Overview of different CNN architectures

- We will ground the evolution on ImageNet Large-Scale Visual Recognition Object Challenge (ILSVRC)
- Training set of 1.2M ($732 - 1300$ training samples per class) labeled images from 1000 categories
- 50K validation set and 100K test set
- Evaluation metric: Top-5 error rate

AlexNet

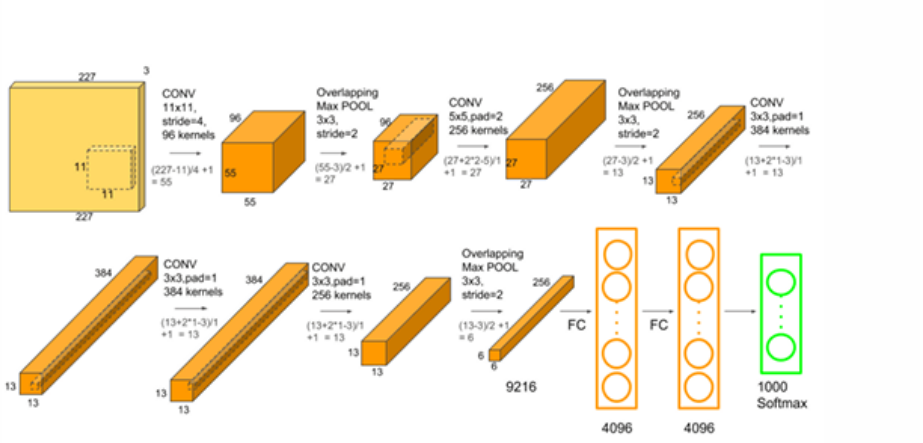
1. 8-layer CNN: 5 Conv layers, 3 FC layers
2. 227×227 input
3. Max pooling, ReLU nonlinearity, LRN (not used anymore now) LRN (Local Response Normalization)

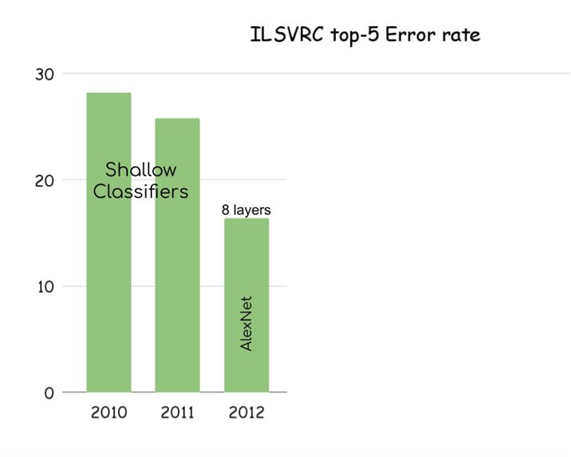
ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

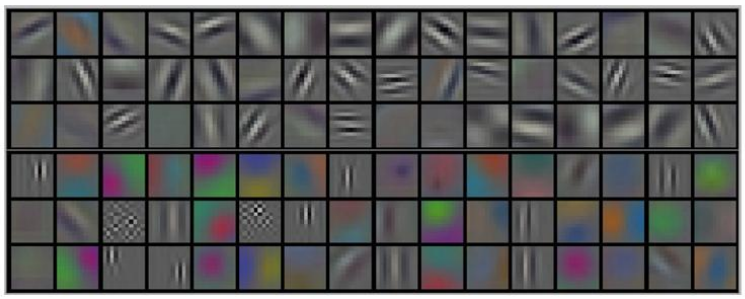
ImageNet has about **1.2 million labeled training images** across **1000 categories**.

There are **50,000 validation images** and **100,000 test images**.

AlexNet (2012)







Visualizing the 11x11 filters learned by AlexNet

ZFNet (2013)

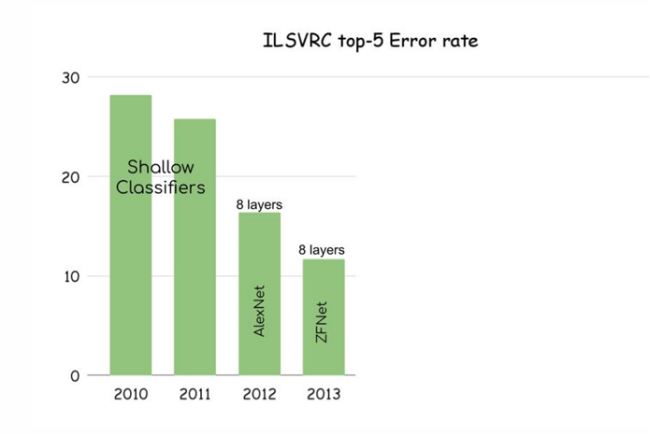
How ZFNet improved AlexNet:

1. Better hyperparameters in the early layers

    - AlexNet's first convolution used a large filter of 11×11 with stride 4 (meaning the filter moves 4 pixels at a time).
    - ZFNet replaced this with a smaller filter of 7×7 with stride 2.
      This smaller filter captures more detailed local features and slides more smoothly across the image, improving feature extraction early on.

2. Changes in convolutional layer sizes

    - AlexNet's Conv layers 3, 4, and 5 had 384, 384, and 256 filters respectively.
    - ZFNet increased them to 512, 1024, and 512 filters respectively, allowing the network to learn more complex and diverse features at these deeper layers.



VGG (2014)

VGG uses multiple small 3×3 filters stacked together.

All conv: 3 ×3, stride:1, pad:1

All max pool: 2 ×2, stride:2 After pooling, double the channels

1. Why only 3×3 Convolutions in VGG?

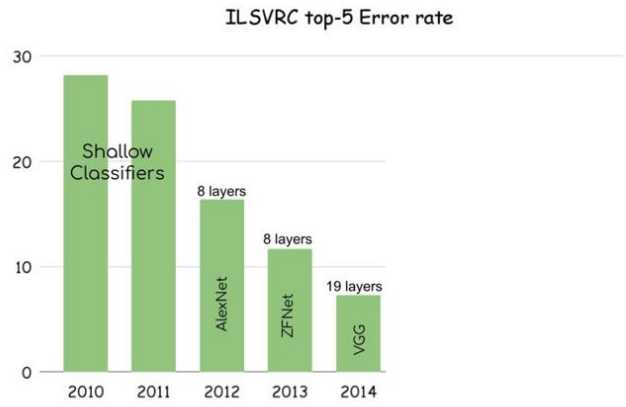
Comparing a single 5×5 convolution to two stacked 3×3 convolutions:

1. Why Only 3×3 Convs?
2. Case-1: Conv$(5 \times 5,\ C \to C)$
   - Parameters: $C \times C \times 5 \times 5 = 25C^2$
   - Flops: $C \times H \times W \times C \times 5 \times 5 = 25C^2HW$

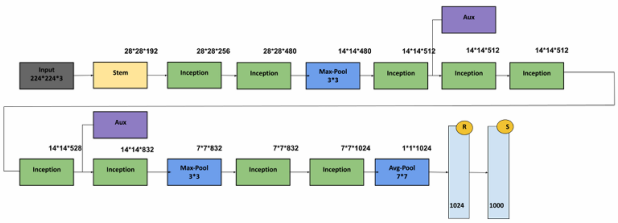
1. Case-2: Conv$(3 \times 3,\ C \to C)$ and Conv$(3 \times 3,\ C \to C)$
   - Parameters: $2 \times C \times C \times 3 \times 3 = 18C^2$
   - Flops: $2 \times C \times H \times W \times C \times 3 \times 3 = 18C^2HW$

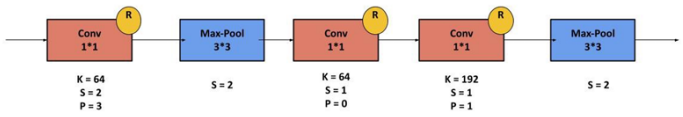
Parameters: Two 3×3 conv layers have fewer parameters ($18C^2$) compared to one 5×5 conv ($25C^2$).

1. Halving the spatial dimensions (max pooling) and doubling the channels $\to$ computational cost is unchanged
2. Case-1: $C \times 2H \times 2W$, Conv $(3 \times 3,\ C \to C)$
   - Memory: 4CHW, parameters: $9C^2$, Flops: $36HWC^2$
3. Case-2: $2C \times H \times W$, Conv $(3 \times 3,\ 2C \to 2C)$
   - Memory: 2CHW, parameters: $36C^2$, Flops: $36HWC^2$



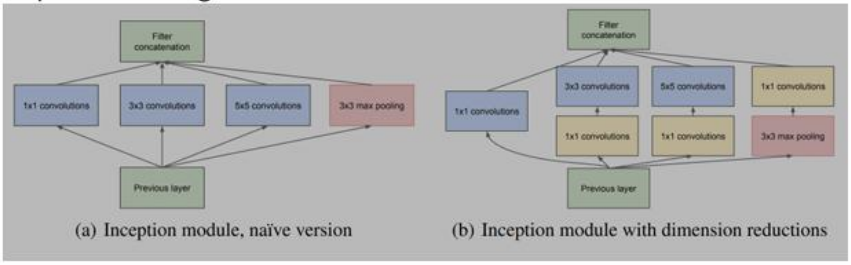
GoogLeNet (2014)

1. Efficiency was the focus of design
2. Reduce the parameters, memory and the compute requirements (towards deployment)
3. 

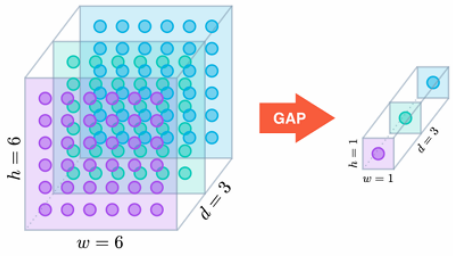
1. Stem architecture at the early stage → aggressive down-sampling
2. 

3. From 224×224 to 28×28
   - GoogLeNet: Compute - 7.5MB, parameters - 124K, and MFlops - 418
   - VGG-16: Compute - 42.9MB (5.7X), parameters - 1.1M (8.9X), and MFlops - 7485 (17.8X)

1. Inception module: unit with parallel branches
2. Repeated through the architecture



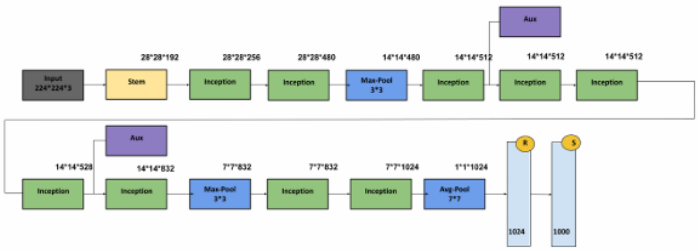
1. Global Average Pooling (GAP) layer
2. Flattening results in huge weight matrices → GoogLeNet introduces GAP layer
3. Collapses the spatial dimensions by computing the average (kernel size = spatial dimensions of the last conv layer)



GoogLeNet (2014)

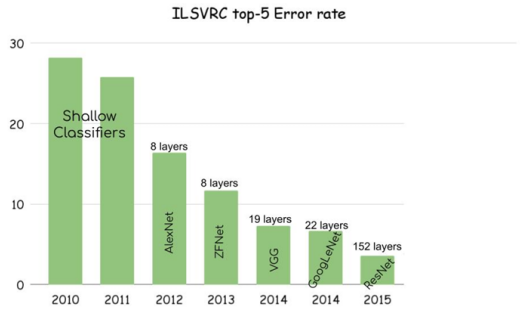
1. No more fully connected layers
2. One linear layer to predict the classification scores (feather light!)

1. Auxiliary classifiers
2. Training using the gradients at the end of the network didn't work well (too deep, gradient propagation was not robust)
3. Hack: add auxiliary classifiers at intermediate locations to receive loss/gradients



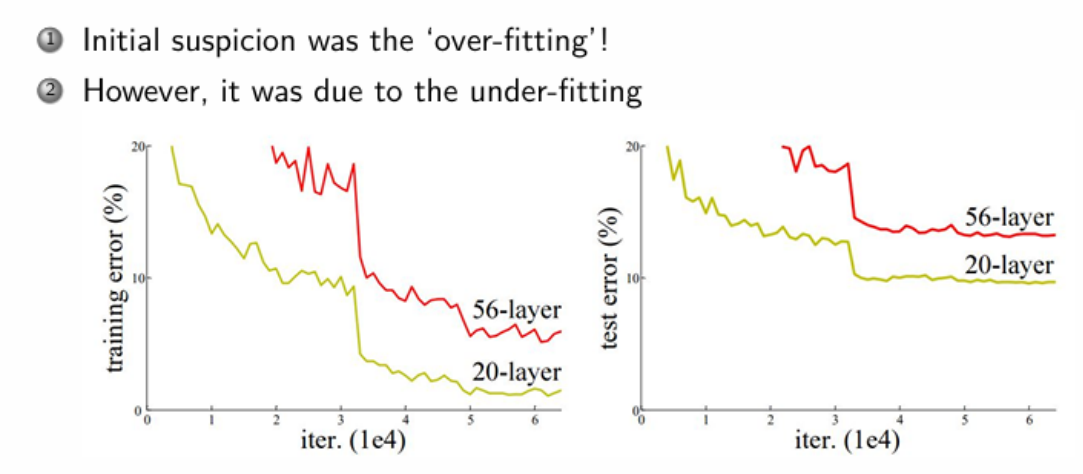
ResNet

1. Very important time for the DNNs
   - Batch Normalization happened
   - Depth increased by an order $(10 \rightarrow 150+)$
   - ILSVRC error almost halved from that of 2014

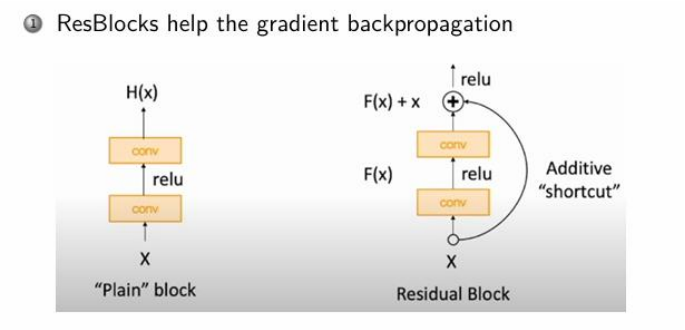


Training Deeper CNNs

When training the “deeper” CNNs, people observed that they were worse than shallow ones



1. Deeper CNNs should easily emulate the shallow ones (extra layers could learn identity function)
2. This is not the case → some issue in the optimization!
3. Work on the architecture so that learning identity function gets easier with additional layers



Plain Block (left)
The network tries to learn the mapping:

$$H(x)$$

Problem during training:

- Gradients must pass through **every layer**
- When the network becomes very deep:
    - gradients **shrink (vanish)**
    - learning becomes difficult

So optimization becomes hard.

Residual Block (right)

Now the block learns:

$$F(x)$$

Final output:

$$H(x) = F(x) + x$$

The shortcut (skip connection) directly adds the input to the output.

1. ResNet is a stack of Resblocks
2. Inspire from VGG and GoogLeNet
3. Simple and regular design like VGG: each resblock has two 3×3 Conv
4. Network has stages: first block of each stage halves the resolution and doubles the channels
5. Aggressive stem in the beginning (downsamples by 4X before the start of the resblocks)
6. Eliminates the FC layers via GAP

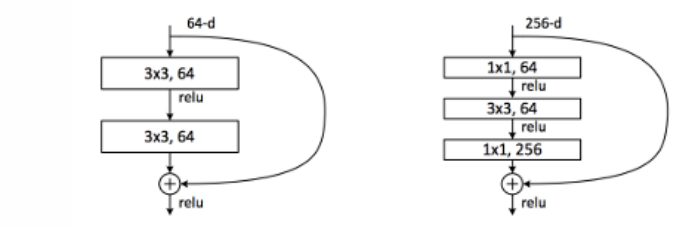
1. ResNet-34
   - Stem: 1 Conv
   - Stage-1 (C=64): 3 resblocks (6 Conv)
   - Stage-2 (C=128): 4 resblocks (8 Conv)
   - Stage-3 (C=256): 6 resblocks (12 Conv)
   - Stage-4 (C=512): 3 resblocks (6 Conv)
   - Linear
   - Top-5 error: 8.58 and GFlop: 3.6 (VGG: 9.6 and 13.6 respectively)

Bottleneck Residual Block

When networks become **very deep (50+ layers)**, computation becomes large.

So ResNet introduced **Bottleneck Blocks**.

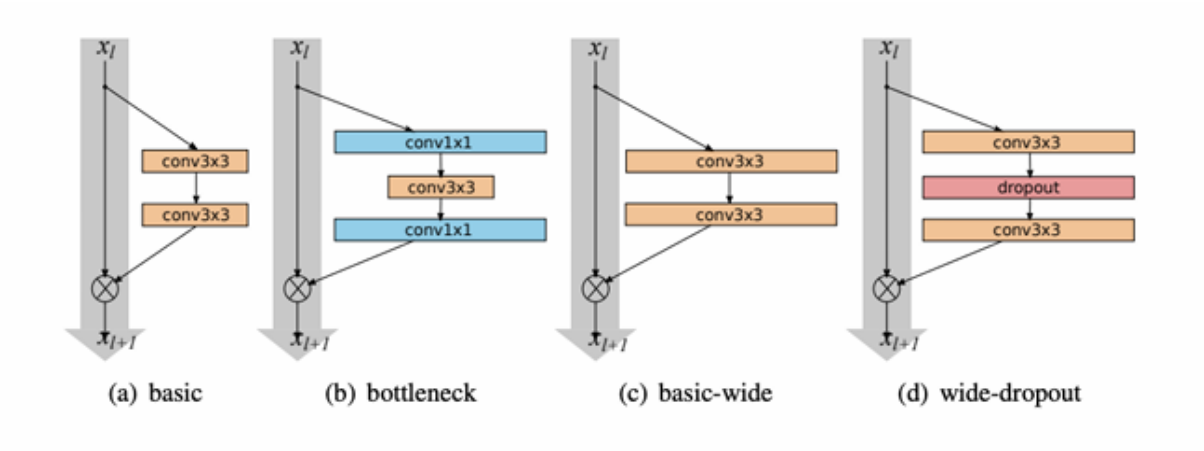
1. Bottlneck Residual block



- Resnet-34 becomes ResNet-50 if we replace the plain resblocks with bottleneck ones
- More blocks at each stage result in ResNet-101 and Resnet-152 architectures

Wide Residual Networks

- Builds on ResNets
- Argues that 'residual connections' are more important than 'depth.'
- Uses wider residual blocks (more filters)
- 16-layer WRN performs the same as a 1000-layer thin deep network
- (comparable parameters)
- Trading width for depth is computationally more efficient



Here is a **very short explanation of each architecture** from your slides.

---

1 ResNeXt (2017)

ResNeXt

- Extension of **ResNet**.
- Uses **parallel convolution paths inside a residual block**.
- Increases **width (cardinality)** instead of only depth.
- Similar idea to **Inception modules**.

2 Stochastic Depth (2016)

- During training, **some residual blocks are randomly skipped**.
- Skipped blocks use **identity mapping**.
- Benefits:

    - faster training
    - reduces vanishing gradients
    - acts as **regularization**
- During **inference**, the **full network is used**.

3 DenseNet (2017)

DenseNet

- Every layer is connected to **all later layers**.
- Output of each layer is **reused by future layers**.
- Benefits:
    - better gradient flow
    - strong feature reuse
    - reduces vanishing gradients.

4 MobileNet (2017)

MobileNet

- Designed for **mobile and embedded devices**.
- Uses **Depthwise Separable Convolution**:
    - Depthwise convolution
    - 1×1 pointwise convolution
- Reduces **computation and model size**.

---

5 Squeeze-and-Excitation Networks (2018)

Squeeze-and-Excitation Network

- Improves CNN features by **learning channel importance**.
- Steps:
    - **Squeeze** → Global Average Pooling
    - **Excitation** → learn weights for channels
- Reweights feature maps to emphasize **important channels**.

---

6 Post-2018 Trend

Focus shifted from **very deep networks → efficient networks**.

Examples:

- **EfficientNet**
- **ShuffleNet**

Also introduced:

- **Neural Architecture Search (NAS)** → automatically design neural networks.